

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

M Shakthi Sridevi
24011101063
AI DS A2

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

<https://github.com/Shakthii27/Deep-Learning-Lab-1/> .

7. Experimental Results

Task 1: Dataset Exploration

First Five Samples

Variance	Skewness	Curtosis	Entropy	Class
3.62160	8.6661	-2.8073	-0.44699	0
4.54590	8.1674	-2.4586	-1.46210	0
3.86600	-2.6383	1.9242	0.10645	0
3.45660	9.5228	-4.0112	-3.59440	0
0.32924	-4.4552	4.5718	-0.98880	0

Table 1: First five samples of the Banknote Authentication dataset

Dataset Dimensions

$$1372 \times 5$$

The dataset contains **1372 samples** and **5 columns** (4 features and 1 target class).

Missing Values

Feature	Missing Values
Variance	0
Skewness	0
Curtosis	0
Entropy	0
Class	0

Table 2: Missing values in the dataset

No missing values are present in the dataset.

Descriptive Statistics

Statistic	Variance	Skewness	Curtosis	Entropy	Class
Count	1372	1372	1372	1372	1372
Mean	0.4337	1.9224	1.3976	-1.1917	0.4446
Std	2.8428	5.8690	4.3100	2.1010	0.4971
Min	-7.0421	-13.7731	-5.2861	-8.5482	0
25%	-1.7730	-1.7082	-1.5750	-2.4135	0
50%	0.4962	2.3197	0.6166	-0.5867	0
75%	2.8215	6.8146	3.1793	0.3948	1
Max	6.8248	12.9516	17.9274	2.4495	1

Table 3: Descriptive statistics of the dataset

Task 2: Exploratory Data Analysis

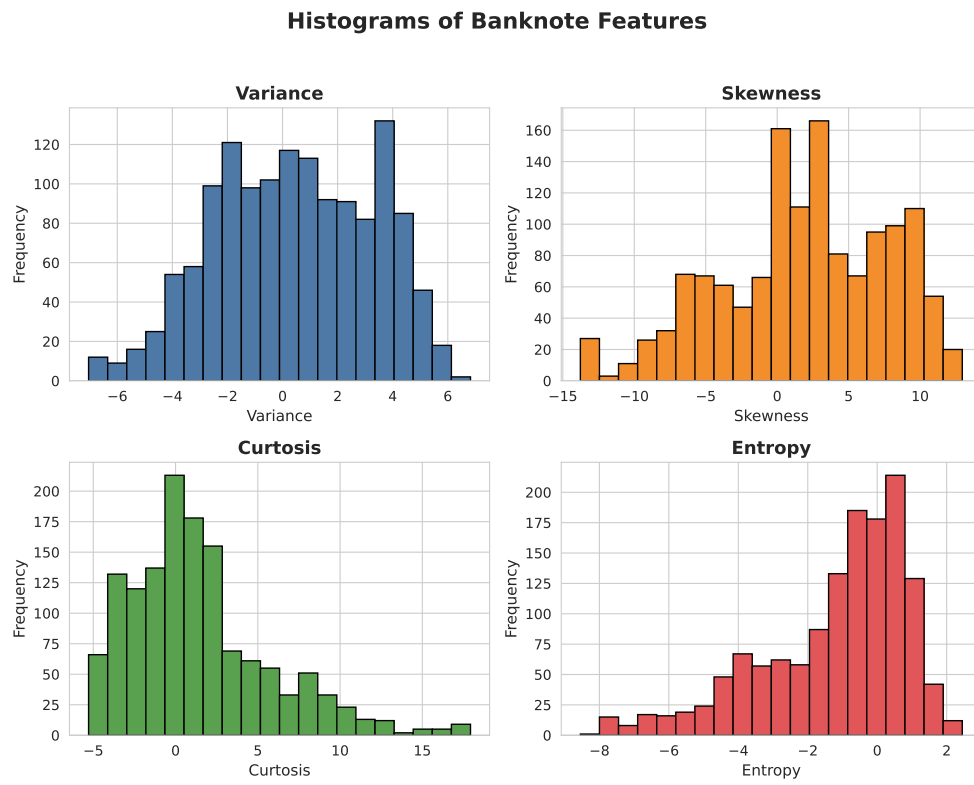


Figure 1: Histograms of all features

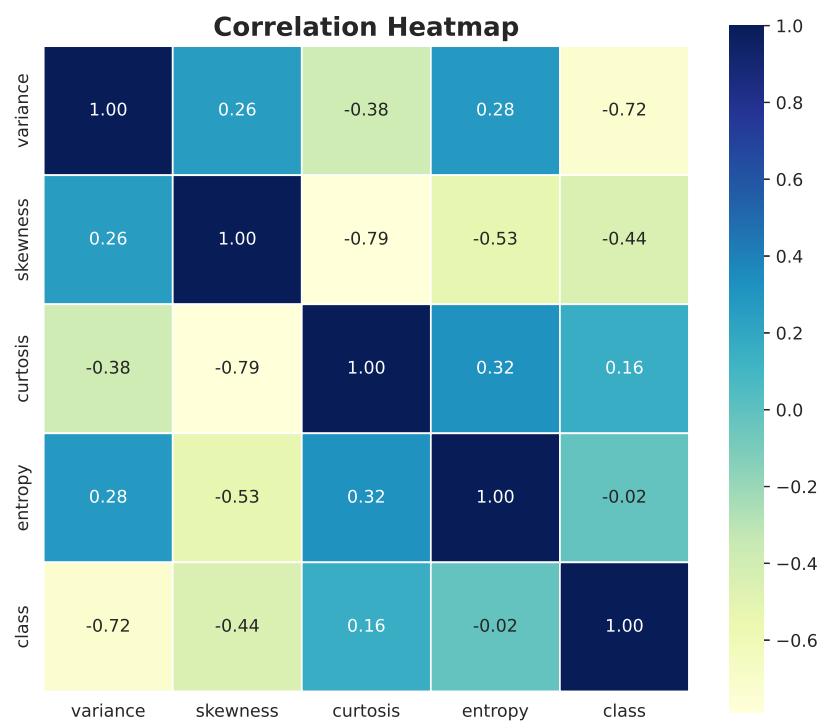


Figure 2: Correlation heatmap

Scatter Plots

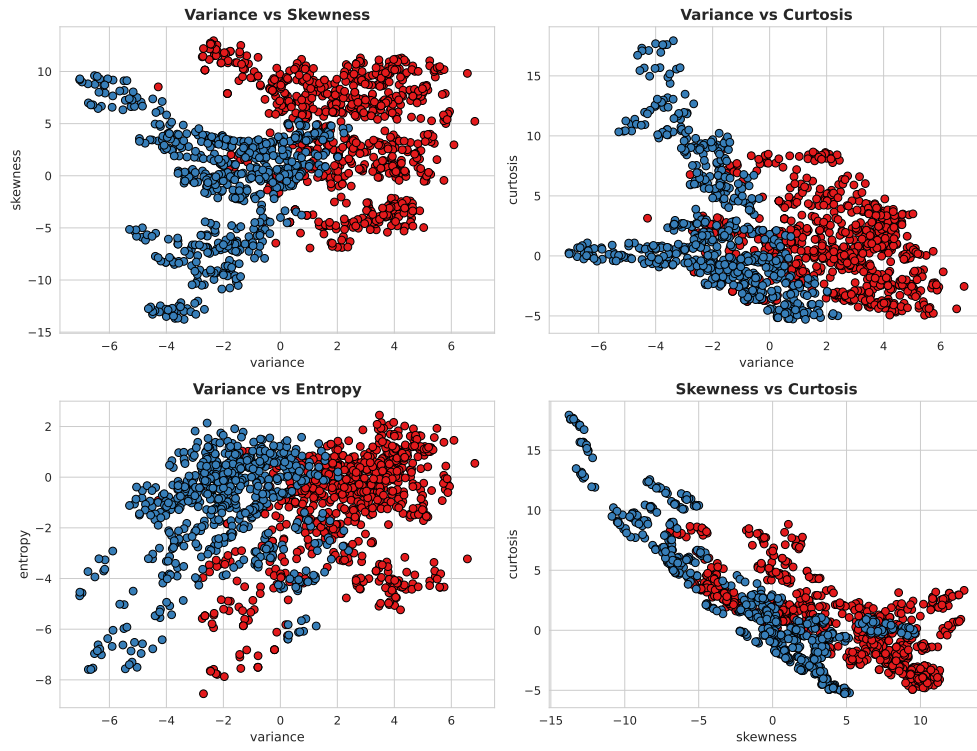


Figure 3: Scatter plot of the dataset

Boxplots of Banknote Features

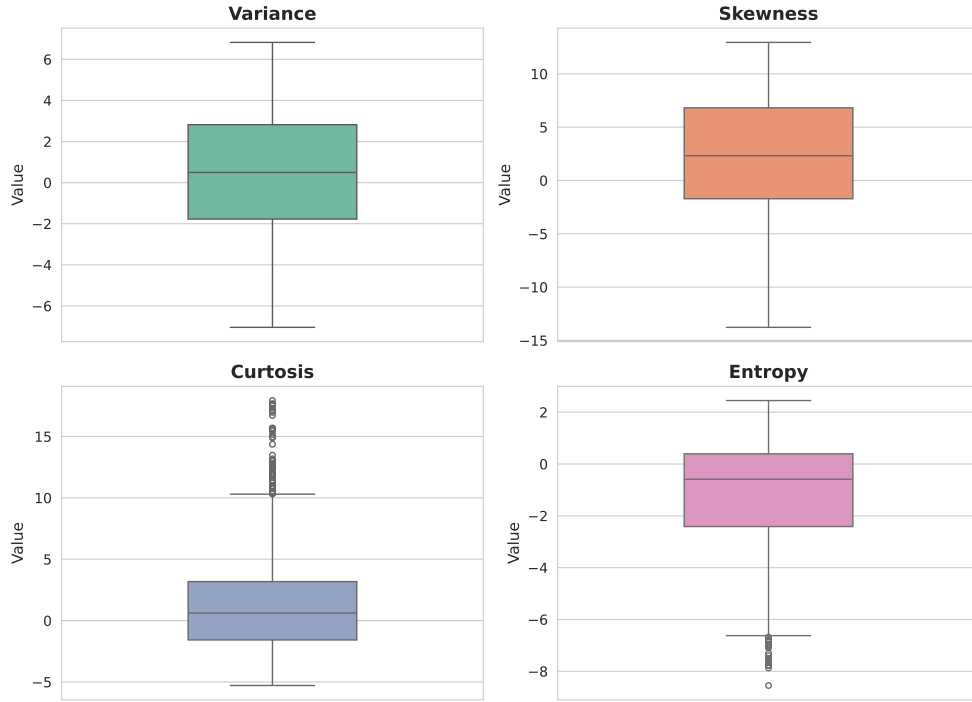


Figure 4: Box plots of all features

Task 3: Data Preprocessing

First Five Rows of Normalized Features

Variance	Skewness	Kurtosis	Entropy
0.769004	0.839643	0.106783	0.736628
0.835659	0.820982	0.121804	0.644326
0.786629	0.416648	0.310608	0.786951
0.757105	0.871699	0.054921	0.450440
0.531578	0.348662	0.424662	0.687362

Table 4: First five rows after Min-Max normalization

Training Set Shape

$$X_{train} : (1097, 4), \quad y_{train} : (1097,)$$

Testing Set Shape

$$X_{test} : (275, 4), \quad y_{test} : (275,)$$

Task 4: Perceptron Model Training

Training completed successfully.

0.0.1 Final Weights

$[-0.212646, -0.223446, -0.224733, -0.006686]$

Final Bias

0.29

Prediction Results

First 10 Actual Labels

$[0, 1, 0, 1, 1, 1, 0, 0, 0, 1]$

First 10 Predicted Labels

$[0, 1, 0, 1, 1, 1, 0, 0, 0, 1]$

Task 5: Training Progress

Epoch	Misclassified Samples	Bias
1	171	0.07
2	101	0.10
3	43	0.11
4	43	0.12
5	57	0.13
6	43	0.14
7	30	0.14
8	49	0.15
9	28	0.15
10	43	0.16
11	24	0.16
12	18	0.16
13	33	0.17
14	30	0.17
15	24	0.17
16	39	0.18
17	26	0.18
18	22	0.18
19	24	0.18
20	27	0.19

Table 5: Training progress over 20 epochs

The number of misclassified samples decreased significantly over the training epochs, indicating that the Perceptron model successfully learned the decision boundary.

Task 6: Model Evaluation

Performance Metrics

Metric	Value
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-Score	98.39%

Table 6: Performance metrics of the Perceptron model

Confusion Matrix

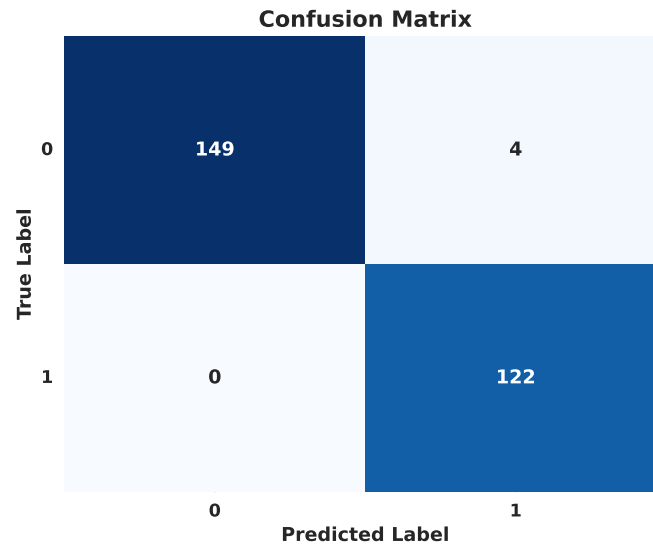


Figure 5: Confusion matrix of the trained Perceptron model

8. Generated Plots with Interpretation

1. Training Error vs Epoch

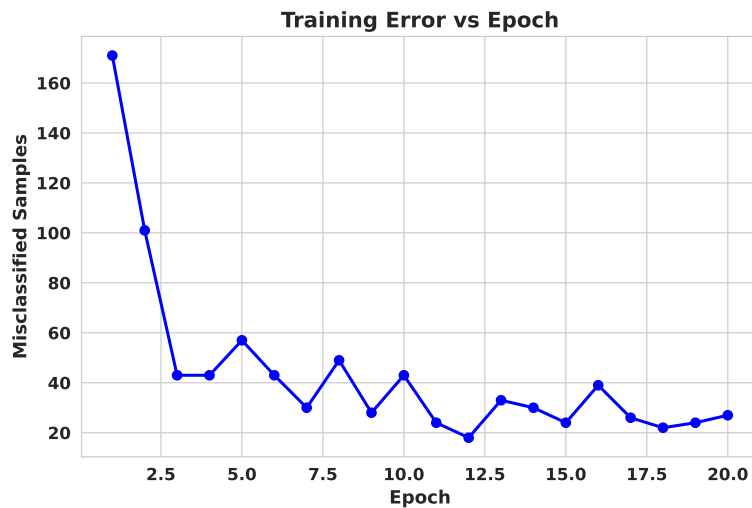


Figure 6: Training Error vs Epoch

The number of misclassified samples decreases significantly during the initial epochs and gradually stabilizes as training progresses. This indicates that the Perceptron successfully learns the decision boundary and converges over time.

2. Weight Evolution

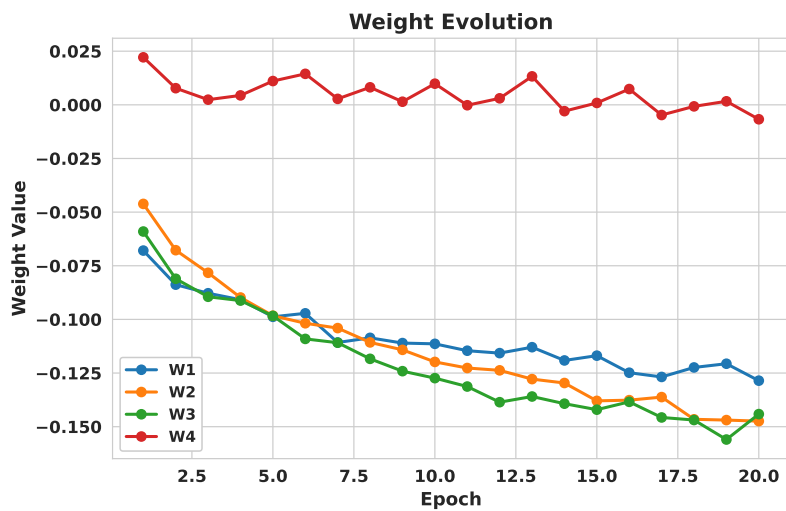


Figure 7: Evolution of Weights During Training

The weights are updated in every epoch based on the classification errors. As training progresses, the weight values gradually stabilize, indicating that the model has converged to an optimal solution.

3. Bias Evolution

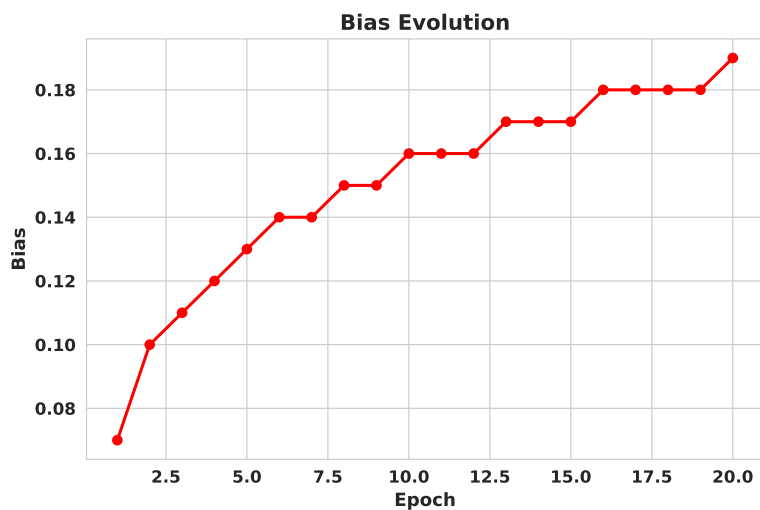


Figure 8: Bias Evolution During Training

The bias increases gradually throughout the training process before becoming stable. This shows that the learning algorithm adjusts the decision boundary until the classification error is minimized.

4. Learning Rate Comparison

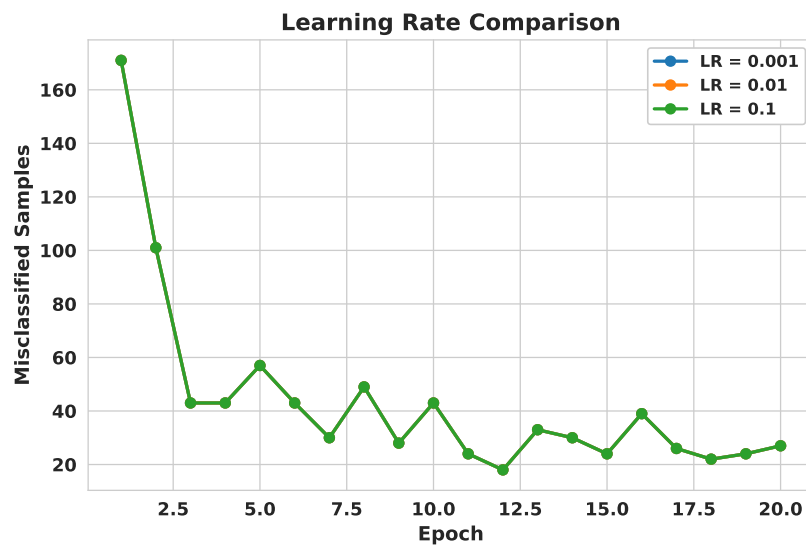


Figure 9: Comparison of Different Learning Rates

Different learning rates affect the speed and stability of convergence. A moderate learning rate provides faster convergence with fewer oscillations, while very small or very large learning rates either slow down learning or produce unstable updates.

5. Decision Boundary

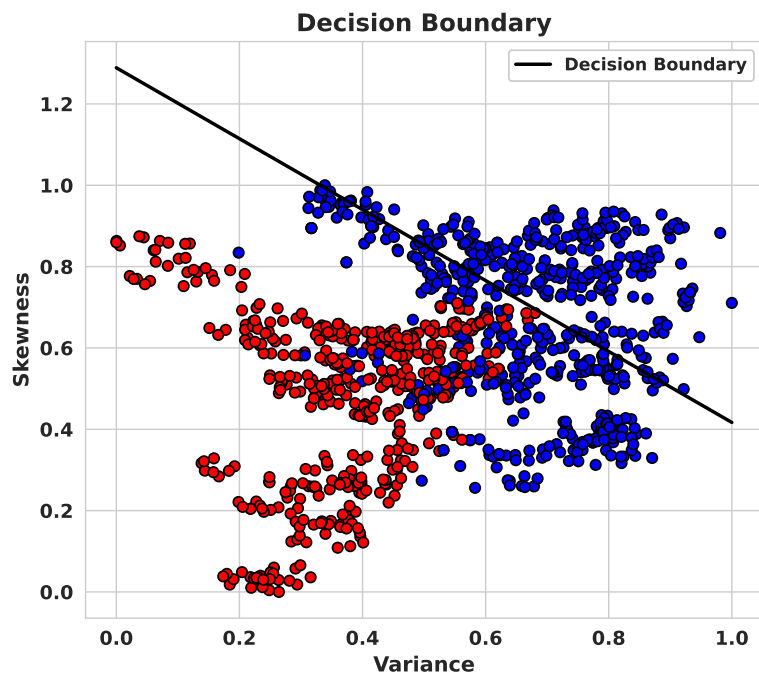


Figure 10: Decision Boundary Learned by the Perceptron

Interpretation: The decision boundary clearly separates the two classes with very few misclassified samples. This demonstrates that the Perceptron is effective for linearly separable datasets such as the Banknote Authentication dataset.

9. Performance Tables

Training Summary

Epoch-wise Learning

Parameter	Value
Dataset Size	1372 samples, 5 attributes
Train/Test Split	1097 training samples (80%), 275 testing samples (20%)
Learning Rate	0.01
Epochs	20
Final Weights	$[-0.2126, -0.2234, -0.2247, -0.0067]$
Final Bias	0.29
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-score	98.39%

Table 7: Training Summary

Epoch	Errors	Weight 1	Weight 2	Bias
1	171	-0.0679	-0.0462	0.07
2	101	-0.0838	-0.0677	0.10
3	43	-0.0878	-0.0783	0.11
4	43	-0.0908	-0.0898	0.12
5	57	-0.0988	-0.0983	0.13
6	43	-0.0972	-0.1018	0.14
7	30	-0.1108	-0.1041	0.14
8	49	-0.1086	-0.1107	0.15
9	28	-0.1110	-0.1142	0.15
10	43	-0.1114	-0.1198	0.16
11	24	-0.1146	-0.1227	0.16
12	18	-0.1157	-0.1237	0.16
13	33	-0.1130	-0.1278	0.17
14	30	-0.1191	-0.1296	0.17
15	24	-0.1169	-0.1380	0.17
16	39	-0.1248	-0.1376	0.18
17	26	-0.1268	-0.1362	0.18
18	22	-0.1224	-0.1466	0.18
19	24	-0.1207	-0.1469	0.18
20	27	-0.1286	-0.1474	0.19

Table 8: Epoch-wise Learning Progress

10. Discussion

The Perceptron model was successfully implemented from scratch and tested on the Banknote Authentication dataset. After preprocessing and training, the model achieved an accuracy of 98.55%, showing that it was able to classify genuine and forged banknotes correctly in most cases. The training error decreased over the epochs, while the weights and bias gradually stabilized, indicating that the model converged successfully.

The generated plots further showed that the learning process was stable and effective. The confusion matrix contained only a few misclassified samples, and the comparison of different learning rates demonstrated that an appropriate learning rate helps the model converge faster. Overall, the results confirm that the Perceptron performs well on linearly separable data.

11. Conclusion

In this experiment, a Perceptron classifier was implemented from scratch using Python and evaluated on the Banknote Authentication dataset. The complete machine learning workflow, including data preprocessing, training, testing, and performance evaluation, was successfully carried out.

The model achieved high accuracy and strong evaluation metrics, demonstrating that the Perceptron is an effective algorithm for binary classification when the data is linearly separable. This experiment also provided a clear understanding of how the Perceptron learning algorithm updates its weights and bias to improve classification performance.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.